Project Report
Design and Analysis of Algorithms

ANUVAB DE SIDDHARTO DAS

This report details the solution for the following problem, its analysis and its implementation. The programme files associated with this project are available on GitHub.

1 Problem Statement

The following problem was selected for this project.

> In organization ABC, the employees have an underlying hierarchy, where the CEO is at the top and every other employee has exactly one manager. An employee can be the manager to one or more employees. Any employee is familiar with the work of her manager and the works of her immediate sub-ordinates. It was decided to give away gift coupons for all employees (including the CEO) as New Year presents. By mistake, the gift coupons ordered were of three different denominations: 1000, 2000 and 3000. It was decided that every employee will receive a gift coupon of denomination different from her manager. However, if an employee receives a gift coupon of larger denomination than her manager, she is sure to be given a negative rating by her manager in a fit of jealousy. Design an algorithm to decide how to distribute gift coupons among employees so that the number of employees who gets a negative rating is minimized.

2 Interpretation

The organisation may be represented as a tree $\mathscr{A}$, with its nodes representing the persons in the organisation. The tree is rooted at the node representing the CEO. The children of any node in the tree represent the sub-ordinates of the person represented by it, if there are any. This can be done since every person other than the CEO is sub-ordinate to exactly one other person.

We are given three labels: 1000, 2000 and 3000. These have the natural numerical ordering. A *distribution of gift coupons among employees* is a vertex labelling of $\mathscr{A}$ with these labels such that no two adjacent vertices are assigned the same label. Given such a labelling, for each edge in the tree (or equivalently, each pair of a person and another person sub-ordinate to him), we say that this edge incurs a penalty if and only if the label on the superior vertex is numerically less than the label on the inferior vertex.

We are asked to find a *distribution of gift coupons among employees* such that the number of edges that incur a penalty is minimised.

3 Solution

The structure of the problem is inductive in nature. Loosely speaking, to minimise the number of penalties incurred in a tree, we ought to minimise the penalties in each immediate subtree keeping in mind the edges which connect the immediate subtrees to the root. Therefore, our first guess should be dynamic programming algorithm on the nodes of the trees, working up from the bottom of the tree. This guess turns out to be right.

We assume that the tree representing the organisation is constructed and already given to us. The children of any node are assumed to be ordered.

Algorithm 1

Given an iterable set of symbols C and a rooted tree T with vertex set V, perform a walk on

`T` in post-order and attach to each node `v` in `V` an integer array `v.bad` indexed by `C` and an array `v.nextset` indexed by `C` the elements of which are themselves arrays the same length as the number of children of `v` containing elements of `C`, in the following manner. Iterating through `C`, for each `curr` in `C`, perform the following steps.

1. Initialise the `c`th element of `v.bad[curr]` to 0.
2. Iterating through the indices `cidx` of the set `v.children` of children of the node `v`, do the following.
   a) Initialise `min` to ∞, and `mincoup` to *empty*.
   b) Iterate through each element `comp` in `C`, skipping an iteration when `curr` = `comp`, and do the following.
      i. Let `angry` be 1 if `curr`<`comp`, 0 otherwise.
      ii. If `angry`+`v.children[cidx].bad[comp]` < `min`, set `min` to `angry`+`v.children[cidx].bad[comp]`, and set `mincoup` to `comp`.
   c) Add `min` to `v.bad[curr]` and set `v.nextset[curr][cidx]` to `mincoup`.

Let `t` be the apical vertex of `T`. Select a `coupon` in `C` such that `t.bad[coupon]` is minimised and let it be `sel`. Start with the node `t` and label it with `sel`. For each index `cidx` in the indices of the set `t.children`, label `t.children[cidx]` with `t.nextset[sel][cidx]` and recurse.

4 Implementation

One practical challenge in the implementation is to take a rooted tree as input in the first place. Let us consider the following. For the purposes of what follows, the children of any node in a rooted tree are linearly ordered.

Definition 1. *Given a rooted tree $\mathcal{T}$, the ith immediate subtree of $\mathcal{T}$ is the subtree of $\mathcal{T}$ rooted at the ith child of the apical vertex of $\mathcal{T}$ and shall be denoted by $\mathcal{T}_i$.*

Definition 2. *Given a rooted tree $\mathcal{T}$, $\Delta(\mathcal{T})$ is the finite sequence the elements of which are the respective numbers of children of the vertices of $\mathcal{T}$ when arranged in pre-order.*

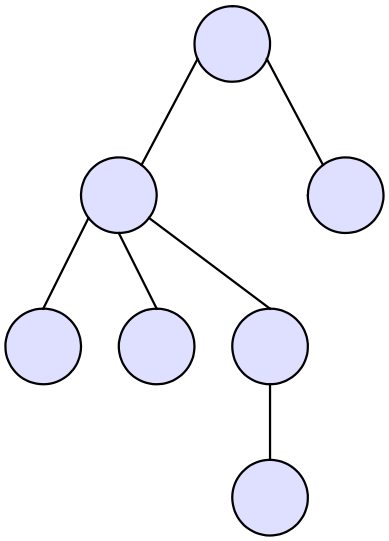


Figure 1. A rooted tree.

Thus, for a tree $\mathcal{T}$ as illustrated in Figure 1, $\Delta(\mathcal{T}) = (2,3,0,0,1,0,0)$.

Proposition 1. *Given a rooted tree $\mathcal{T}$, using $\Delta(\mathcal{T}) = \{\delta_n\}_{n=1}^N$ form another sequence $\Sigma(\mathcal{T}) = \{\sigma_n\}_{n=1}^N$ as follows:*

$$\begin{aligned} \sigma_1 &= -\delta_1 \\ \sigma_{n+1} &= \sigma_n - \delta_{n+1} + 1 \end{aligned} \tag{1}$$

Then, $\sigma_n \leqslant 0$ for every n and $\sigma_n = 0$ precisely when $n = N$.

Proof. By strong induction on N.

Base case. We have $N = 1$. Therefore, the tree only has one vertex with 0 children. So, $\sigma_1 = -\delta_1 = 0$. This completes the base case.

Inductive step.

Suppose the proposition is proved for all N up to k. We have the case $N = k+1$. We must have $\delta_1 > 0$ otherwise we would have $N = 0$. Then, by the definition of $\Delta(\mathcal{T})$, $\Delta(\mathcal{T})$ has $\delta_1 + 1$ components:

$$\delta_1 \left| \underbrace{\delta_2, \ldots, \delta_{1+n_1}}_{\Delta(\mathcal{T}_1)} \right| \underbrace{\delta_{1+n_1+1}, \ldots, \delta_{1+n_1+n_2}}_{\Delta(\mathcal{T}_2)} \left| \cdots \right| \underbrace{\delta_{1+n_1+n_2+\cdots+n_{\delta_1-1}+1}, \ldots, \delta_{1+n_1+\cdots+n_{\delta_1}}}_{\Delta(\mathcal{T}_{\delta_1})}$$

where $\mathcal{T}_i$ is the ith immediate subtree of the tree $\mathcal{T}$.

For each i, $1 \leqslant i \leqslant \delta_1$, let $\{\sigma_j^{(i)}\}_{j=1}^{n_i}$ be the sequence $\Sigma(\mathcal{T}_i)$. Consider some n, $2 \leqslant n \leqslant N$. Write $n = n_0 + n_1 + \cdots + n_{k-1} + u$ where $1 \leqslant k \leqslant \delta_1$ and $1 \leqslant u \leqslant n_k$. Then,

$$\begin{aligned} \sigma_n &= -\delta_1 + \sum_{l=1}^{k-1} \sum_{j=1}^{n_l} \left(-\delta_{\sum_{m=0}^{l-1} n_m + j} + 1\right) + \sum_{j=1}^{u} \left(-\delta_{\sum_{m=0}^{k-1} n_m + j} + 1\right) \\ &= -\delta_1 + \sum_{l=1}^{k-1} (1 + \sigma_{n_l}^{(l)}) + (1 + \sigma_j^{(k)}) \\ &= -\delta_1 + k + \sum_{l=1}^{k-1} \sigma_{n_l}^{(l)} + \sigma_j^{(k)} \end{aligned}$$

Using the induction hypothesis, $\sigma_{n_l}^{(l)} = 0$ for each l such that $1 \leqslant l \leqslant k-1$ and $\sigma_j^{(k)} \leqslant 0$. Thus,

$$\sigma_n = -\delta_1 + k + \sigma_j^{(k)} \leqslant -\delta_1 + k \leqslant 0$$

For equality, we need $k = \delta_1$ and $\sigma_j^{(k)} = 0$. Again, by the induction hypothesis, this happens exactly when $j = n_k = n_{\delta_1}$. Therefore, $\sigma_n \leqslant 0$ for every n and $\sigma_n = 0$ exactly when $n = n_0 + n_1 + \cdots + n_{\delta_1} = N$. This completes the inductive step and the proof. □

Definition 3. *Two rooted trees $\mathcal{T}$ and $\mathcal{T}'$ are said to be* isomorphic *if there is a one-to-one map F of the vertex set V of $\mathcal{T}$ onto the vertex set V' of $\mathcal{T}'$ which has the following two properties:*

1. *(u,v) is an edge in $\mathcal{T}$ if and only if $(F(u), F(v))$ is an edge in $\mathcal{T}'$.*
2. *For any vertex u and two children v and w of u, if v appears before w in the ordering of children of u in $\mathcal{T}$, then $F(v)$ appears before $F(w)$ in the ordering of children of $F(u)$ in $\mathcal{T}'$.*

Then F is said to be an isomorphism *of $\mathcal{T}$ onto $\mathcal{T}'$.*

To obtain a simple method to accept rooted trees as input, we wish to prove the following statement.

Proposition 2. *Suppose there are two rooted trees $\mathcal{T}$ and $\mathcal{T}'$ such that $\Delta(\mathcal{T}) = \Delta(\mathcal{T}') = \{\delta_n\}_{n=1}^{N}$, then $\mathcal{T}$ and $\mathcal{T}'$ are isomorphic as rooted trees.*

We would then have established that taking $\Delta(\mathcal{T})$ as input is equivalent to having $\mathcal{T}$ as input. First, we need to settle some lemmas.

Lemma 1. *The image u_2 of the apical vertex u_1 of a rooted tree $\mathcal{T}_1$ under an isomorphism F onto a rooted tree $\mathcal{T}_2$ is the apical vertex of the latter.*

Proof. Suppose not. Then, there is a vertex v in $\mathcal{T}_2$ such that (v, u_2) is an edge in $\mathcal{T}_2$. Since F is an isomorphism, $(F^{-1}(v), u_1)$ is an edge in $\mathcal{T}_1$. This is in contradiction with the hypothesis that u_1 is the apical vertex of $\mathcal{T}_1$. □

Lemma 2. *Let F be a one-to-one map of the vertex set of a rooted tree $\mathcal{T}$ onto the vertex set of a rooted tree $\mathcal{U}$ such that the following hold:*

1. *The image under F of the apical vertex t of $\mathcal{T}$ is the apical vertex u of $\mathcal{U}$.*
2. *The number of children of t in $\mathcal{T}$ is the same as the number of children of u in $\mathcal{U}$.*
3. *For every i, $F|_{V_i}$ is an isomorphism of $\mathcal{T}_i$ onto $\mathcal{U}_i$, where V_i is the vertex set of $\mathcal{T}_i$.*

Then, F is an isomorphism of $\mathcal{T}$ onto $\mathcal{U}$.

Proof. F is clearly one-to-one and onto. We need to prove the two conditions in Definition 3.

Let (v, w) be an edge in $\mathcal{T}$. If (v, w) is in $\mathcal{T}_i$ for some i, then using condition 3 above, $(F(v), F(w))$ is an edge in $\mathcal{U}_i$, and therefore, an edge in $\mathcal{U}$. Otherwise, $v = t$ and w is the apical vertex of $\mathcal{T}_i$ for some i. Using condition 3 and Lemma 1, $F(w)$ is the apical vertex of $\mathcal{U}_i$, so $F(w)$ is a child of u. Therefore, $(F(v), F(w)) = (u, F(w))$ is an edge in $\mathcal{U}$. Symmetrically, whenever $(F(v), F(w))$ is an edge in $\mathcal{U}$, (v, w) is an edge in $\mathcal{T}$. Thus, the first condition in Definition 3 is satisfied.

From the previous paragraph it is clear that for any two children v and w of t, if v appears before w, then $F(v)$ appears before $F(w)$ among the children of u. Now, fix any vertex t' in $\mathcal{T}$ other than $\mathcal{T}$. Then, t' is in $\mathcal{T}_i$ for some i. Say v and w are children of t' and v appears before w among the children of t'. Since $F|_{\mathcal{T}_i}$ is an isomorphism, $F(v)$ appears before $F(w)$ among the children of $F(t')$. Thus, the second condition in Definition 3 is satified, and F is an isomorphism. □

We are ready to prove Proposition 2.

Proof of Proposition 2. To prove that two rooted trees are isomorphic, we will use Lemma 2. The proof is by strong induction on the length of $\Delta(\mathcal{T})$.

Base case. If the length of $\Delta(\mathcal{T})$ is 1, then clearly, the mapping on the vertex set of $\mathcal{T}$ that maps the only vertex of $\mathcal{T}$ to the only vertex of $\mathcal{T}'$ is an isomorphism of $\mathcal{T}$ onto $\mathcal{T}'$. This completes the base case.

Inductive step. Suppose the theorem is proved for all N up to k. We have the case $N = k+1$. We must have $\delta_1 < 0$. We already have that the number of children of the apical vertex of $\mathcal{T}$ is the same as the apical vertex of $\mathcal{T}'$, viz., δ_1. We know that we can decompose $\Delta(T) = \Delta(\mathcal{T}')$ as follows.

$$\delta_1 \Big| \underbrace{\delta_2, \ldots, \delta_{1+n_1}}_{\Delta(\mathcal{T}_1)} \Big| \underbrace{\delta_{1+n_1+1}, \ldots, \delta_{1+n_1+n_2}}_{\Delta(\mathcal{T}_2)} \Big| \cdots \Big| \underbrace{\delta_{1+n_1+n_2+\cdots+n_{\delta_{1-1}}+1}, \ldots, \delta_{1+n_1+\cdots+n_{\delta_1}}}_{\Delta(\mathcal{T}_{\delta_1})}$$

Looking in $\mathcal{T}'$,

$$\delta_1 \Big| \underbrace{\delta_2, \ldots, \delta_{1+n_1'}}_{\Delta(\mathcal{T}_1')} \Big| \underbrace{\delta_{1+n_1'+1}, \ldots, \delta_{1+n_1'+n_2'}}_{\Delta(\mathcal{T}_2')} \Big| \cdots \Big| \underbrace{\delta_{1+n_1'+n_2'+\cdots+n_{\delta_{1-1}}'+1}, \ldots, \delta_{1+n_1'+\cdots+n_{\delta_1}'}}_{\Delta(\mathcal{T}_{\delta_1}')}$$

Set up a recurrence starting at the second index in $\{\delta_n\}_{n=1}^{N}$.

$$\begin{aligned}\sigma_1^{(1)} &= -\delta_2\\ \sigma_{n+1}^{(1)} &= \sigma_n^{(1)} - \delta_{n+2} + 1\end{aligned}$$

Using Proposition 1, the first occurence of 0 in $\{\sigma_n^{(1)}\}_n$ is at the position n_1 since $\{\sigma_n^{(1)}\}_n$ begins with $\Sigma(\mathcal{T}_1)$. Since it also begins with $\Sigma(\mathcal{T}_1')$, $n_1 = n_1'$. Thus, $\Delta(\mathcal{T}_1) = \Delta(\mathcal{T}_1')$.

This way, step-by-step, once we have shown $n_m = n_m'$ and $\Delta(\mathcal{T}_m) = \Delta(\mathcal{T}_m')$, we can set up another recurrence:

$$\begin{aligned}\sigma_1^{(m+1)} &= -\delta_{1+n_1+n_2+\cdots+n_m+1}\\ \sigma_{n+1}^{(m+1)} &= \sigma_n^{(m+1)} - \delta_{1+n_1+n_2+\cdots+n_m+(n+1)} + 1\end{aligned}$$

Using Proposition 1, the first occurence of 0 in $\{\sigma_n^{(m+1)}\}_n$ is at the position n_{m+1} since $\{\sigma_n^{(m+1)}\}_n$ begins with $\Sigma(\mathcal{T}_{m+1})$. Since it also begins with $\Sigma(\mathcal{T}_{m+1}')$, $n_{m+1} = n_{m+1}'$, and therefore, $\Delta(\mathcal{T}_{m+1}) = \Delta(\mathcal{T}_{m+1}')$.

Using the induction hypothesis, we have an isomorphism F_i of $\mathcal{T}_i$ onto $\mathcal{T}_i'$ for each i such that $1 \leqslant i \leqslant \delta_1$. Let t be the apical vertex of $\mathcal{T}$ and t' be the apical vertex of $\mathcal{T}'$. Define the map F of the vertex set of $\mathcal{T}$ onto the vertex set of $\mathcal{T}'$ as follows.

$$F(v) = \begin{cases} t' & \text{if } v = t\\ F_i(v) & \text{if } v \in \mathcal{T}_i \text{ for some } i \text{ such that } 1 \leqslant i \leqslant \delta_1\end{cases}$$

From Lemma 2, F is an isomorphism of $\mathcal{T}$ onto $\mathcal{T}'$. This completes the inductive step and the proof. □

This method of input is implemented in the code using the function `struct tree *read_tree (FILE *input)` in `tree.c`:

```
struct tree *read_tree (FILE *input)
{
        int _get_num(FILE *);

        int multi, i, coupon;
        struct tree *prod;
        bool failed;

        if ((multi = _get_num(input)) < 0)
                return NULL;
        prod = malloc(sizeof(struct tree));
        prod->multi = multi;
        prod->children = multi ? malloc(sizeof(struct tree *) * multi) :
                                 NULL;
        failed = false;
        for (i = 0; i < multi; i++) {
                if (failed) {
                        prod->children[i] = NULL;
                        continue;
                }
                if ((prod->children[i] = read_tree(input)) == NULL)
                        failed = true;
        }
        for (coupon = 0; coupon < COUPON_COUNT; coupon++) {
                prod->solv[coupon].bad = 0;
                prod->solv[coupon].nextset = NULL;
```

```
        }
        if (failed) {
                destroy_tree(prod);
                return NULL;
        }
        return prod;
}
```

Briefly, $\Delta(\mathcal{T}) = \{\delta_n\}_{n=1}^{N}$ for some tree $\mathcal{T}$ is read by first reading δ_1 and then recursively reading in $\Delta(\mathcal{T}_i)$ for $1 \leqslant i \leqslant \delta_1$.

`struct tree` is a C `struct` defined as follows in `employees.h`:

```
struct tree {
        size_t multi;
        struct tree **children;
        struct suborder {
                int bad;
                int *nextset;
        } solv[COUPON_COUNT];
};
```

`size_t multi` is the number of children of the apical vertex, and `struct tree **children` represents an array of pointers to the immediate subtrees. In the programme, coupons are represented by integers ranging from zero to *COUPON_COUNT*−1. *COUPON_COUNT* is fixed at 3 in `employees.h`.

`void destroy_tree(struct tree *work)` is a destructor for a dynamically allocated tree (not shown here for the sake of brevity), and `int _get_num (FILE *input)` is an auxiliary function that returns whitespace-separated unsigned integers from the input one after the other:

```
int _get_num (FILE *input)
{
        int num;
        int c;

        while (isspace(c = getc(input)))
                ;
        if (!isdigit(c))
                return -1;
        num = c - '0';
        while (isdigit(c = getc(input)))
                num = 10 * num + c - '0';
        ungetc(c, input);
        return num;
}
```

Algorithm 1 itself is implemented using `void postorder_act (struct tree *work, void (*action)(struct tree *))` and `void compute_order (struct tree *work)` in `employees.c`:

```
void postorder_act (struct tree *work, void (*action)(struct tree *))
{
        size_t i;
```

```
        for (i = 0; i < work->multi; i++)
                postorder_act(work->children[i], action);
        (*action)(work);
}

void compute_order (struct tree *work)
{
        size_t cidx;
        struct tree *child;
        int curr, comp;
        int min, min_coupon;
        int more_bad;
        int angry;

        if (work->multi > 0) {
                for (curr = 0; curr < COUPON_COUNT; curr++) {
                        work->solv[curr].bad = 0;
                        work->solv[curr].nextset = malloc(
                            sizeof(int) * work->multi);
                        for (cidx = 0; cidx < work->multi; cidx++) {
                                child = work->children[cidx];
                                min = -1;
                                angry = 0;
                                for (comp = 0; comp < COUPON_COUNT;
                                    comp++) {
                                        if (curr == comp) {
                                                angry++;
                                                continue;
                                        }
                                        more_bad =
                                            child->solv[comp].bad +
                                            angry;
                                        if (min < 0 || more_bad < min) {
                                                min = more_bad;
                                                min_coupon = comp;
                                        }
                                }
                                work->solv[curr].bad += min;
                                work->solv[curr].nextset[cidx] =
                                    min_coupon;
                        }
                }
        }
}
```

`postorder_act` is used to perform a post-order walk on a given tree and `compute_order` specifies the action to perform at each node as prescribed by Algorithm 1. The output is generated using the function `void write_tree (struct tree *work, int spec)` and the auxiliary function `void _write_tree_r (struct tree *work, int spec, const wchar_t *prefix, bool fin)`:

```
void write_tree (struct tree *work, int spec)
```

```
{
        void _write_tree_r(struct tree *, int, const wchar_t *, bool);
        _write_tree_r(work, spec, NULL, true);
}

void _write_tree_r (struct tree *work, int spec, const wchar_t *prefix,
    bool fin)
{
        size_t cidx;
        wchar_t *newpre;
        printf("%ls%ls%d\n", prefix == NULL ? L"" : prefix,
            prefix == NULL ? L"" :
                fin        ? L"\u2514\u2500\u2500␣" :
                             L"\u251C\u2500\u2500␣",
            1000 * (spec + 1));
        if (work->multi) {
                if (prefix == NULL)
                        newpre = L"";
                else {
                        newpre = malloc(
                            sizeof(wchar_t) * (wcslen(prefix) + 5));
                        wcscpy(wcpcpy(newpre, prefix),
                            fin ? L"\u00A0\u00A0\u00A0␣" :
                                  L"\u2502\u00A0\u00A0␣");
                }
                for (cidx = 0; cidx < work->multi - 1; cidx++)
                        _write_tree_r(work->children[cidx],
                            work->solv[spec].nextset[cidx], newpre,
                            false);
                _write_tree_r(work->children[cidx],
                    work->solv[spec].nextset[cidx], newpre, true);
                if (prefix != NULL)
                        free(newpre);
        }
}
```

`_write_tree_r` recursively outputs a given tree while maintaining a prefix in front of each entry depending on the position and depth of the entry. To wit, before any “final” entry, that is, an entry which is the last child of some vertex, the characters `"\u2514\u2500\u2500␣"` are prefixed, otherwise, the characters `"\u251C\u2500\u2500␣"` are prefixed. Additionally, when recursing to deeper subtrees, if the current tree is rooted at a “final” entry, then `"\u00A0\u00A0\u00A0␣"` is added to the running prefix for padding, otherwise `"\u2502\u00A0\u00A0␣"` is appended. All this is done to match the output of the LINUX utility `tree`. How `_write_tree_r` works with the `nextset` field is explained in the next section.

These components of the programme are wrapped up in `int main ()` to complete the implementation of Algorithm 1.

```
int main ()
{
        void postorder_act(struct tree *, void (*)(struct tree *));
        void compute_order(struct tree *);
```

```
        void write_tree(struct tree *, int);

        struct tree *employees;
        int coupon;
        int min, min_coupon;

        setlocale(LC_ALL, "C.UTF-8");
        employees = read_tree(stdin);
        if (employees == NULL) {
                fprintf(stderr, "cannot convert input to tree\n");
                return 1;
        }
        postorder_act(employees, compute_order);

        min = -1;
        for (coupon = 0; coupon < COUPON_COUNT; coupon++) {
                if (min < 0 || employees->solv[coupon].bad < min) {
                        min = employees->solv[coupon].bad;
                        min_coupon = coupon;
                }
        }
        write_tree(employees, min_coupon);
        destroy_tree(employees);

        return 0;
}
```

5 Proof of correctness

Let us start with some definitions and propositions. Later, we shall interpret the quantities used in Algorithm 1 in terms of these and justify its correctness. We shall work here with the number of coupons any arbitrary integer not less than 2.

Notation 1. *Given a rooted tree $\mathcal{T}$, we shall denote the vertex set of $\mathcal{T}$ by $V(\mathcal{T})$ and the edge set of $\mathcal{T}$ by $E(\mathcal{T})$. The apical vertex of $\mathcal{T}$ shall be denoted by $a(\mathcal{T})$.*

Definition 4. *Given a rooted tree $\mathcal{T}$ and a set C, the set of* distributions *of C in $\mathcal{T}$ is the set of all maps χ of $V(\mathcal{T})$ into C such that whenever $(v,w) \in E(\mathcal{T})$, $\chi(v) \neq \chi(w)$. The set of distributions of C in $\mathcal{T}$ shall be denoted by $\mathscr{H}(\mathcal{T}, C)$.*

Definition 5. *Given a rooted tree $\mathcal{T}$ and a distribution χ of a set C linearly ordered by '$\leqslant$' in $\mathcal{T}$, the* penalty *of χ is the quantity $\sum_{(v,w)\in E(\mathcal{T})} [\chi(v) < \chi(w)]$, and shall be denoted by $B(\chi, \mathcal{T})$.*

Here, $[\cdot]$ stands for the *Iverson bracket*.
Note that we can make always make the following reduction:

$$B(\chi, \mathcal{T}) = \sum_i [\chi(a(\mathcal{T})) < \chi(a(\mathcal{T}_i))] + B(\chi|_{V(\mathcal{T}_i)}, \mathcal{T}_i)$$

Definition 6. *Given a rooted tree $\mathcal{T}$ and $c \in C$ where C is a finite set linearly ordered by '$\leqslant$', define $D(\mathcal{T}, c) = \min\{B(\chi, \mathcal{T}) \mid \chi \in \mathscr{H}(\mathcal{T}, C), \chi(a(\mathcal{T})) = c\}$.*

This quantity follows the following recurrence relation.

Proposition 3. *Given a rooted tree $\mathscr{T}$, a finite set C linearly ordered by '$\leqslant$' and some $c \in C$,*

$$D(\mathscr{T}, c) = \sum_i \min\{[c < d] + D(\mathscr{T}_i, d) \mid d \in C \setminus \{c\}\} \tag{2}$$

Proof. Let $\chi \in \mathscr{H}(\mathscr{T}, C)$ be such that $\chi(a(\mathscr{T})) = c$ and that $B(\chi, \mathscr{T}) = D(\mathscr{T}, c)$ is achieved. Now,

$$\begin{aligned} B(\chi, \mathscr{T}) &= \sum_i \left([c < \chi(a(\mathscr{T}_i))] + B(\chi|_{V(\mathscr{T}_i)}, \mathscr{T}_i)\right) \\ &\geqslant \sum_i \left([c < \chi(a(\mathscr{T}_i))] + D(\mathscr{T}_i, \chi(a(\mathscr{T}_i)))\right) \\ &\geqslant \sum_i \min\{[c < d] + D(\mathscr{T}_i, d) \mid d \in C \setminus \{c\}\} \end{aligned}$$

since $\chi(a(\mathscr{T}_i)) \neq \chi(a(\mathscr{T})) = c$ from the definition of $\mathscr{H}(\mathscr{T}, C)$.

On the other hand, for each i, let $d_i \in C \setminus \{c\}$ be such that $[c < d_i] + D(\mathscr{T}_i, d_i) = \min\{[c < d] + D(\mathscr{T}_i, d) \mid d \in C \setminus \{c\}\}$. For each i, let $\chi_i \in \mathscr{H}(\mathscr{T}_i, C)$ be such that $\chi_i(a(\mathscr{T}_i)) = d_i$ and that $B(\chi_i, \mathscr{T}_i) = D(\mathscr{T}_i, d_i)$. Define the map $\chi : V(\mathscr{T}) \to C$ as follows:

$$\chi(v) = \begin{cases} c & \text{if } v = a(\mathscr{T}) \\ \chi_i(v) & \text{if } v \in \mathscr{T}_i \text{ for some } i \end{cases}$$

Since $\chi_i(a(\mathscr{T}_i)) \neq c$ for every i, $\chi \in \mathscr{H}(\mathscr{T}, C)$. Then, we have,

$$\begin{aligned} \sum_i \min\{[c < d] + D(\mathscr{T}_i, d) \mid d \in C \setminus \{c\}\} &= \sum_i \left([c < \chi_i(a(\mathscr{T}_i))] + B(\chi_i, \mathscr{T}_i)\right) \\ &= \sum_i \left([\chi(\mathscr{T}) < \chi(a(\mathscr{T}_i))] + B(\chi|_{V(\mathscr{T}_i)}, \mathscr{T}_i)\right) \\ &= B(\chi, \mathscr{T}) \\ &\geqslant D(\mathscr{T}, c) \end{aligned}$$

This completes the proof. □

As a corollary, we have the following result.

Corollary 1. *Suppose we are given a rooted tree $\mathscr{T}$, a finite set C ordered by '$\leqslant$' and $c \in C$, and suppose for each i and for each $d \in C \setminus \{c\}$, we are given $\chi_{i,d} \in \mathscr{H}(\mathscr{T}_i, C)$ such that $\chi_{i,d}(a(\mathscr{T}_i)) = d$ and that $B(\chi_{i,d}, \mathscr{T}_i) = D(\mathscr{T}_i, d)$. For each i, let $d_i \in C \setminus \{c\}$ be such that $[c < \chi_i(a(\mathscr{T}_i))] + B(\chi_i, \mathscr{T}_i) = \min\{[c < \chi_{i,d}(a(\mathscr{T}_i))] + B(\chi_{i,d}, \mathscr{T}_i) \mid d \in C \setminus \{c\}\}$, where $\chi_i = \chi_{i,d_i}$.*

Let $\chi : V(\mathscr{T}) \to C$ be defined as follows:

$$\chi(v) = \begin{cases} c & \text{if } v = a(\mathscr{T}) \\ \chi_i(v) & \text{if } v \in \mathscr{T}_i \text{ for some } i \end{cases}$$

Then, $\chi \in \mathscr{H}(\mathscr{T}, C)$ and $B(\chi, \mathscr{T}) = D(\mathscr{T}, c)$.

Proof. Refer to the second part of the proof of Proposition 3. From the definition of χ_i, we have that

$$\begin{aligned} [c < \chi_i(a(\mathscr{T}_i))] + B(\chi_i, \mathscr{T}_i) &= \min\{[c < \chi_{i,d}(a(\mathscr{T}_i))] + B(\chi_{i,d}, \mathscr{T}_i) \mid d \in C \setminus \{c\}\} \\ &= \min\{[c < d] + D(\mathscr{T}_i, d) \mid d \in C \setminus \{c\}\} \end{aligned}$$

From the second part of the proof of Proposition 3,

$$B(\chi, \mathscr{T}) = \sum_i [c < \chi_i(a(\mathscr{T}_i))] + B(\chi_i, \mathscr{T}_i)$$

Thus, using Proposition 3, $B(\chi, \mathcal{T}) = D(\mathcal{T}, c)$. □

Now, in the algorithm, we work on any vertex only after all its children are already dealt with, since we do a post-order walk on the tree. The algorithm computes recurrence (2) for each vertex in the tree and for each coupon c and stores the value in the `bad[c]` field. The algorithm also computes for each coupon c, solutions for d_i as described in Corollary 1 and stores the values in the fields `nextset[c][i]`.

Once $D(\mathcal{T}, c)$ is calculated for each $c \in C$, the problem statement asks us to find $\min_{\chi \in \mathcal{H}(\mathcal{T}, C)} B(\chi, \mathcal{T})$ and a solution $\chi_0 \in \mathcal{H}(\mathcal{T}, C)$ such that $B(\chi_0, \mathcal{T}) = \min_{\chi \in \mathcal{H}(\mathcal{T}, C)} B(\chi, \mathcal{T})$. Clearly, $\min_{\chi \in \mathcal{H}(\mathcal{T}, C)} B(\chi, \mathcal{T}) = \min_{c \in C} D(\mathcal{T}, c)$ and a valid solution χ_0 is given by constructing $\chi \in \mathcal{H}(\mathcal{T}, C)$ such that $B(\chi, \mathcal{T}) = \min_{c \in C} D(\mathcal{T}, c)$ and setting $\chi_0 = \chi$ with as described in Corollary 1, since $\min_{c \in C} D(\mathcal{T}, c) = D(\mathcal{T}, c')$ for some $c' \in C$ and we can construct such a χ with $\chi(a(\mathcal{T})) = c'$.

Once we have calculated for every vertex v of $\mathcal{T}$ and for every $c \in C$, the values of d_i's as described in Corollary 1, we can output the solution χ_0. By Corollary 1 again, to print all the labels, we need to output $\chi_0(a(\mathcal{T})) = c'$ first (this is passed by `main` to `write_tree` and then to `_write_tree_r`) and recursively put the labels given by χ_i for each i as described in Corollary 1. `_write_tree_r` does this while taking care of formatting according to the depth in the tree. Thus, the algorithm works as intended.

6 Complexity analysis

Notation 2. *We shall use n for $\#(V(\mathcal{T}))$ and m for $\#(E(\mathcal{T}))$.*

Note that for a tree, $m = n - 1$.

It is useful to break the working of the programme into four sections as demarcated in `main`:

1. Input
2. Computation
3. Output
4. Garbage collection

`COUPON_COUNT` will be treated as a fixed constant for all estimates that follow.

6.1 Input

`_get_num` consumes $O(1)$ time for each character read. The length of the input is bounded above by the expression $\sum_{v \in V(\mathcal{T})} (\log_{10}(\deg(v) + 1) + 1)$, using the definition of $\Delta(\mathcal{T})$. But

$$\begin{aligned}\sum_{v \in V(\mathcal{T})} (\log_{10}(\deg(v) + 1) + 1) &= n + \sum_{v \in V(\mathcal{T})} \log_{10}(\deg(v) + 1) \\ &\leqslant n + n \log_{10}\left(\frac{n + \sum_{v \in V(\mathcal{T})} \deg(v)}{n}\right) \\ &= n + n \log_{10}(m + n) - n \log_{10}(n)\end{aligned}$$

Thus, `_get_num` consumes $O(n \log(m))$ time over the course of a single run of the programme. To initialise all values, `read_tree` consumes $O(\deg(v))$ for each $v \in V(\mathcal{T})$ apart from the time consumed by `_get_num`. Thus, the total time consumed by `read_tree` is $O(m) + O(n \log(m)) = O(n \log(n))$. However, practically, no number input can exceed `INT_MAX` by the specifications of the programme, so that the total time consumed in `_get_num` is actually $O(n)$. Thus, effectively, the total time consumed during the input phase is $O(n)$.

6.2 Computation

`compute_order` consumes $O(\deg(v))$ time when it is run on any vertex v in the programme. `compute_order` is called recursively on each vertex via `postorder_act` with some $O(1)$ extra time consumed for bookkeeping on each call. Thus, the computation phase consumes $O(n)$ time.

6.3 Output

`_write_tree_r` consumes $O(\deg(v))$ time on each vertex v with some $O(1)$ extra time for bookkeeping. Over all recursive calls, `_write_tree_r` consumes $O(m) = O(n)$ time for output. Calculation of *spec* to pass to `_write_tree_r` via `write_tree` from `main` is done in $O(1)$ time. Thus, the output phase consumes $O(n)$ time.

6.4 Garbage collection

The analysis for `destroy_tree` is similar to that of `read_tree`. (See file `tree.c`.) The time consumed in the garbage collection phase is $O(n)$.